

1. Robot Design and Strategy Overview

Mechanical:

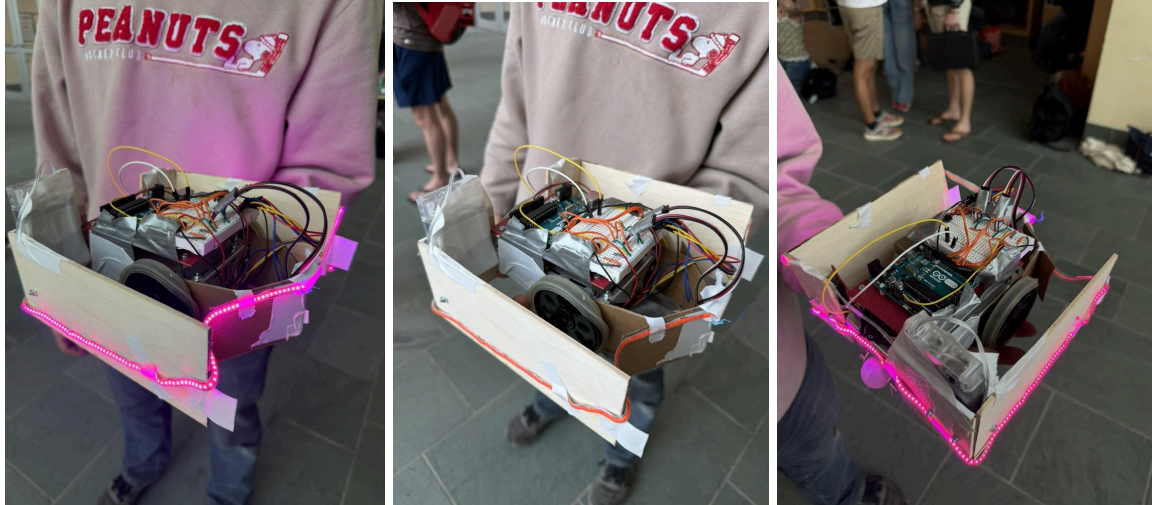
The robot includes the metal chassis that holds all boards, batteries, and sensors. Outside of that, there are two side walls and a black wall mounted onto the chassis that surrounds the robot. For the block collecting part of the design, we wanted to make sure the cubes were always able to enter, so we chose to avoid having any gates that the blocks could get stuck on. Instead, we opted to have two channels for the blocks to be funneled into (each channel made up of the outer side wall and the chassis), and because the robot never drives backward, nothing ever causes them to slide out. The robot also had everything mounted as low to the ground as possible, especially the batteries. This kept the robot's center of mass low, ensuring it was stable and maintained good traction. Another factor to maintain good traction was using non slippery wheel covers. In the front of the robot, we placed a triangle to funnel the cubes to either side of the robot into the channels. This ensured all the blocks went into the right place (and didn't get caught in any of the wheels).

Electrical:

To avoid having too many points of failure, we tried to keep the electrical aspects simple and efficient. We utilized two QTI sensors, one color sensor, and the two continuous rotation servos that were given to us. Each servo was controlled by a H bridge. We put tape around the QTIs and color sensors to ensure other light from the area didn't bleed into them. We also made sure to spend a lot of time tuning the color sensor to make sure it was as accurate as possible. We also added bright LEDs, with the hope of the color messing up the color sensing of other robots.

Software:

The goal of the software design of the robot was to have it always zigzag around the middle line. The robot starts by being hard coded to go straight in the middle and then turn a little. Once it crosses the middle line, the robot would start its zigzag logic. It would drive forward for a certain amount of time after crossing the line, and then turn 90 degrees and drive until it hits the color switch again, and repeat. It knew which way to turn by keeping track of which way it turned last time. The drive forward command was also designed to be a rapidly iterating while loop, where in each iteration it would check to see if the robot was over the black border (through the QTI sensors). This way, anytime the robot hits the black border, it will turn 180 degrees and continue its zig zag. To ensure the robot never strays too far from the middle line (in case it is knocked off course), we count how long it has been driving forward, and if it is too long, it turns 90 degrees back toward the middle. This helps ensure we never get caught in a loop bouncing between the borders in the corner.



2. Design Process Reflection

Our group set clear, high-level goals before approaching the robot's hardware and software design. For the remainder of the semester, we aimed to implement these goals as we progressed through the milestones to prevent a large quantity of work concentrated in the time leading up to the competition.

From the hardware perspective, our robot aimed to conserve mechanical simplicity to prevent issues on competition day. As such, mechanical development progressed quickly between milestones three and four. We prototyped the arm designs using cardboard for rapid iteration and testing. When the cube collecting method using the arms and front splitter was qualified with the cardboard, we moved to the final design utilizing $\frac{1}{8}$ " thick birchwood sheets. This progressed without issue. However, the layout of the breadboard and microcontroller was repeatedly adjusted throughout the course of the semester to achieve updated goals and work with constraints like wiring lengths, battery pack placement, etc.

From the software side, our group scaffolded new, milestone specific code on top of code from previous portions of the course. For example, the same code from the in class lectures generating functions to drive forwards, backwards, and turn was used in our final code. The largest challenge came in milestone 3 when implementing our border detection and correction strategy. Having the QTI's reliably detect the border took multiple lab sessions and assistance from the TA's. Ultimately, the border detection strategy was successful and became a key component of our later competition and center-seeking strategy. Tuning the zig-zag and center-seeking strategy took a significant amount of time. Countless iterations with different turns and path lengths were tested on the competition board until the robot consistently zig-zagged through the middle third of the board containing the blocks. Eventually, we tuned the center-seeking and zig-zag algorithm well enough that almost all cubes were collected within one minute when testing without another robot present.

3. Competition Analysis

Our robot competition started strong, but ended slightly disappointingly. In our first three competitions, our robot performed nominally, collecting blocks and retaining them without issue. Our robot went undefeated through these first three games. In the fourth game, an anomaly seemed to occur: our robot slipped off the side of the board early in the match and was unable to generate enough torque to climb back into play. It is still unclear why the QTI's and control logic did not steer the robot away from the border as it had in testing and in every other match on competition day. Our robot consequently lost round four. In match five, our robot returned to nominal behavior and we won, moving our record to four wins, one loss—a relatively successful history. The last match of the round robin and the first of the single elimination games reversed this trend.

In match six of the round robin our robot did not move when the round began. After the match, we attempted to diagnose the issue, checking wiring, batteries, and possible mechanical issues. However, when we tested it again without any adjustments, it worked without issue. In the single elimination round, our robot failed to move again, leading to our early exit from the competition. After further investigation, we realized a poor connection existed in the jumper cable from the battery pack to the breadboard that must've degraded during the competition day.

Overall, our robot was very successful in accomplishing the competition's mission, and had a high success rate when it was free of electrical issues. The robot's control logic was the highlight of the design. The code effectively and repeatedly navigated us towards the center of the board, zig-zagging through the blocks. Unfortunately, our wiring was not robust enough to handle a long day of competition, and led to our exit from the tournament.

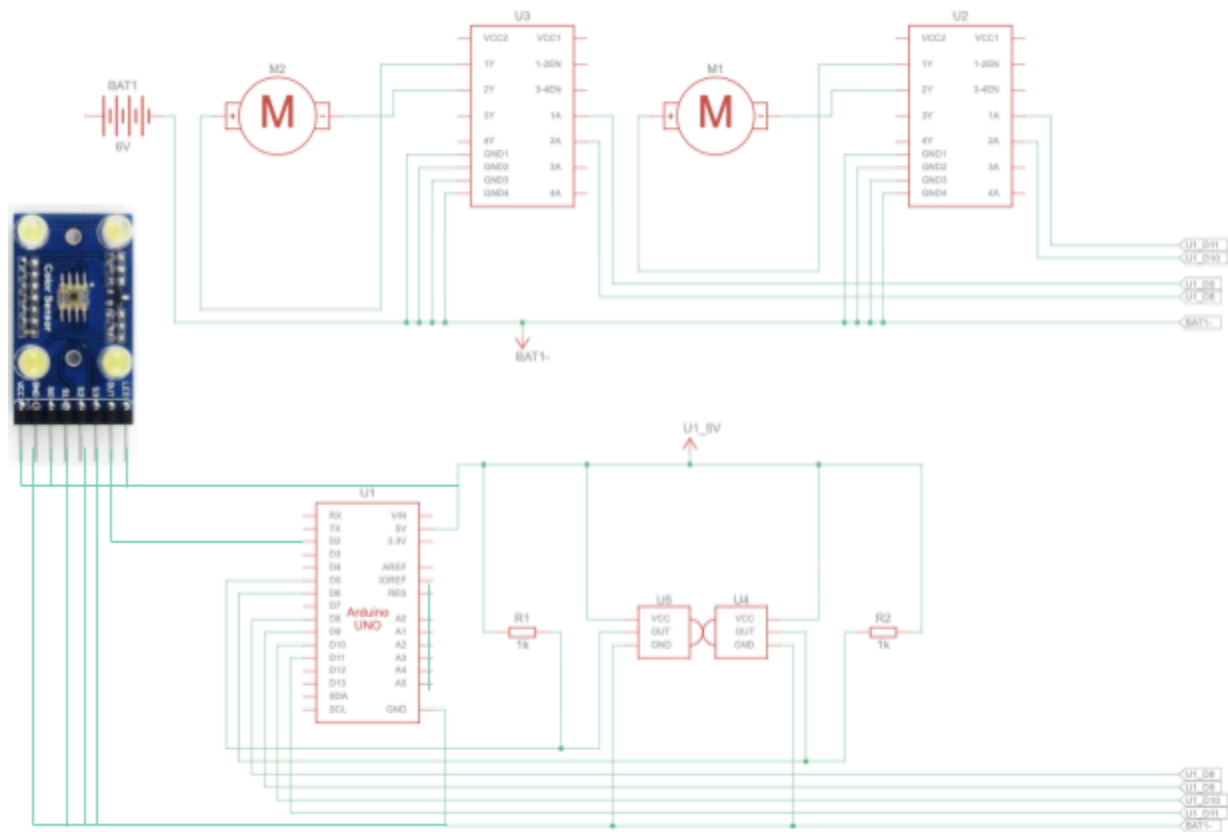
4. Conclusions

Because our robot behaved well in the first three rounds (before our jumper wire became loose), we believe all high level decisions were decent, and would not want to change too much. Our code performed well. While many robots got lost ours was good at staying near the middle which proved to be a good advantage. One thing we would change is the wiring management. Having better wiring management to make it easier in the future to notice loose cables. Something we would recommend to future students is to use GenAI as little as possible. This came in handy for us when de-coding because it was really easy to find and fix bugs.

5. Appendix A

LINE	PART	QUANTITY	LINE PRICE	SUPPLIER / PART NUMBER	NOTES
1	Angle Brackets	10 @ \$0.24 each	\$2.40	McMaster Carr 131353A59	
2	LEDs	1	12.99	B0G6F6Q3LH	Personal item found
3	Cardboard	1			Scavenged from household; priced referencing Amazon
4	Birchwood	1	\$7.95	Amazon	Cut to size in machine shop
5	Duct Tape	N/A			Found in MAE3780 lab

6. Appendix B



7. Appendix C

CAD files used to manufacture the robot arms. See line 4 of the BOM.



Mass properties of short_arm
 Configuration: Default
 Coordinate system: -- default --
 Density = 0.02 pounds per cubic inch
 Mass = 0.02 pounds
 Volume = 1.03 cubic inches
 Surface area = 17.99 square inches



Mass properties of Part1

Configuration: Default

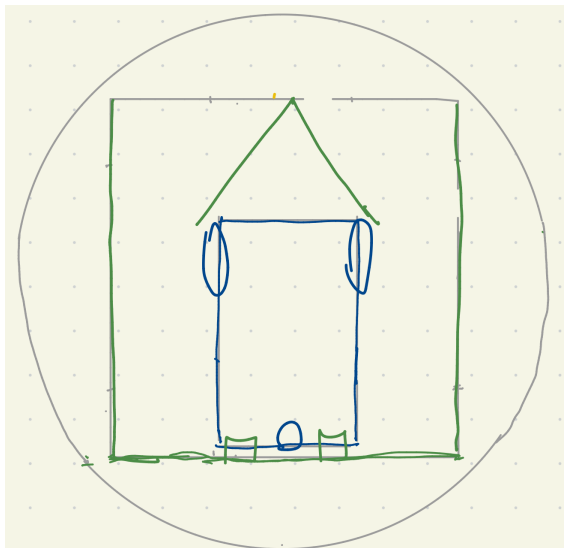
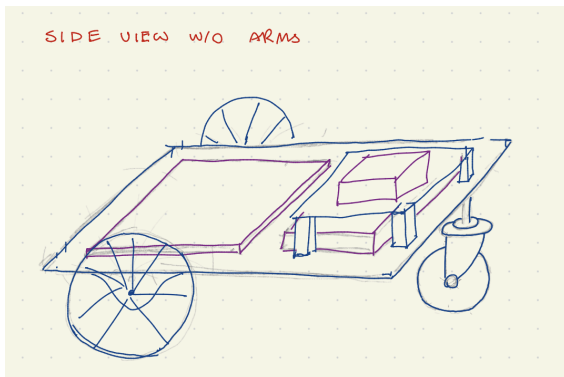
Coordinate system: -- default --

Density = 0.02 pounds per cubic inch

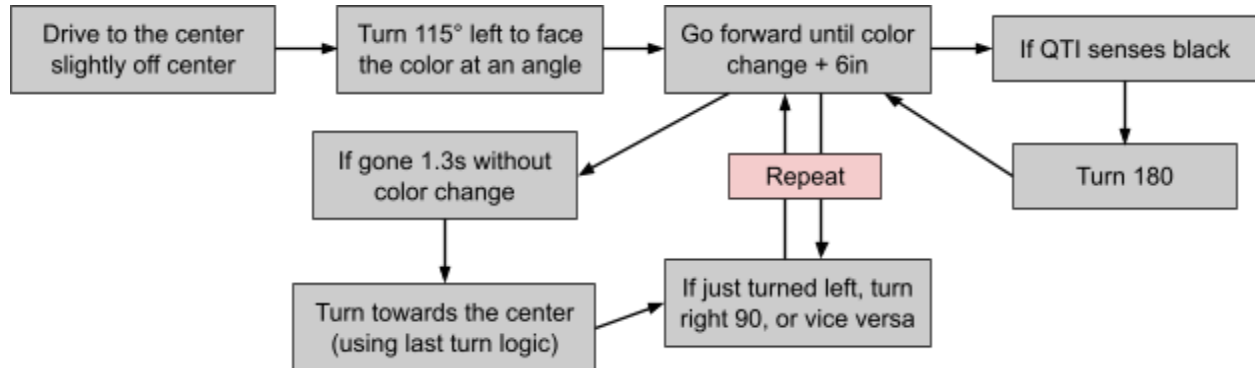
Mass = 0.05 pounds

Volume = 2.62 cubic inches

Surface area = 44.52 square inches



8. Appendix D



9. Appendix E

```
int period;
int timer;
int counter2 = 0;
// --- ISR: an interrupt function that resets and reads the timer value
ISR(PCINT2_vect) {

    // resets the timer to zero on a rising edge,
    if (PIND & 0b00000100) { //rising edge
        TCNT1 = 0;
    }
    // and stores the timer value in a variable ("timer") on a falling edge (or vice versa).
    else { //falling edge
        timer = TCNT1;
    }
}

// --- initColor: initializes the interrupt and timer
void initColor() {
    // a. Initialize your I/O pins
    DDRD = 0b00001000; //output in PD7

    // b. Initialize my pin change interrupt
    PCICR = 0b00000100; //PCINT2
    PCMSK2 = 0b00000000; //set to 0 for now
    // c. Initialize your timer
    TCCR1A = 0b00000000;
    TCCR1B = 0b00000001; //prescalar of 1
}
```

```

// --- getColor: calculates the period of the sensor output wave and returns it as a
variable
int getColor() {
    // a. Enable the specific bit for your Pin Change Interrupt
    PCMSK2 = 0b000000100;
    // b. Add a short delay (5 ~ 10 milliseconds)
    _delay_ms(1);
    // c. Disable the specific bit for your pin change interrupt
    // (to prevent further interrupts until you call getColor again)
    PCMSK2 = 0b000000000;
    // d. Return the period in units of time
    long per = timer / 8;
    /*Serial.print("period:");
    Serial.print(per);
    Serial.print("if statement:");
    Serial.println(per > 150);*/
    if (per > 150) { //ADJUST TO BETTER TUNE BLUE/YELLOW TUNING
        //Serial.print("BLUEUUUUU");
        return 1;
    }
    //Serial.print("YELLOW");
    return 0;
}

int main(void) { // setup code that only runs once
    // set pins 8 and 9 as outputs for "left" motor
    DDRB = 0b00001111;
    initColor();
    Serial.begin(9600);
    PORTD = 0b01101000;
    sei();
    int startColor = getColor();
    int color = startColor;
    int lastTurn = 1;
    int turnCounter = 0;

    //hard coded start to get to the set up approach to the center
    drive_forward_delay(800);
    stop();
    turn_left_delay(200);

```



```

stop();
drive_forward_delay(10);

while (color == startColor) {
    if (PIND & 0b01000000) { //LEFT SEES
        stop();
        lastTurn = 1;
        turn_right_delay(400); // turn right 90 degrees
        stop();
        drive_forward_delay(10); //ADJUST IF STOPPING AFTER BLACK
    } else if (PIND & 0b00100000) {
        stop();
        lastTurn = 0;
        Serial.println("Detected right, turning left");
        turn_left_delay(400); // turn right 90 degrees
        stop();
        drive_forward_delay(10);
    }
    color = getColor();
}
//detected the change in color, turn left to position a start zigzag
_delay_ms(500);
stop();
turn_left_delay(540);
drive_forward_break(1);
// start zigzagging
lastTurn = 1;
color = getColor();
while(1) {
    counter2= counter2 + 1;
    drive_forward_break(1);
    if (color != getColor()) {
        color = getColor();
        bool free = drive_forward_break(850); //forward a little for the zigzag
        if (free) {
            if (lastTurn == 1) { //if we just turned right, turn left
                turn_right_delay(450);
                stop();
                lastTurn = 0;
                counter2 = 0;
            } else { //if we just turned left, turn right
                turn_left_delay(490);
            }
        }
    }
}

```

```

        stop();
        lastTurn = 1;
        counter2 = 0;
    }
}
}
//if we have gone 1300ms since hitting a color change, turn back towards the center
//as we have likely hit a section without color changes
if(counter2>=1300){
    if (lastTurn == 1) {
        turn_right_delay(450);
        stop();
        lastTurn = 0;
    } else {
        turn_left_delay(490);
        stop();
        lastTurn = 1;
    }
    counter2 = 0;
}
}
}

void stop() {
    PORTB = (PORTB & 0b11110000) | 0b00000000;
    _delay_ms(50);
}
//drive forward but 180 if you see black
bool drive_forward_break(int t) {
    bool free = 1;
    for (int i = 0; i < t; i++) {
        PORTB = (PORTB & 0b11110000) | 0b00000110;
        if (PIND & 0b01000000 || PIND & 0b00100000) {
            // WE ARE AT THE END
            turn_right_delay(950);
            PORTB = (PORTB & 0b11110000) | 0b00000110; // drive forward
            counter2 = 0;
            free = 0;
            //_delay_ms(400);
            //stop(); // we dont want it to stop now
            break;

```

```

    }

    _delay_ms(1); // wait for t milliseconds
}

return free;
}

// functions with built-in delay
void drive_forward_delay(int t) {
    PORTB = (PORTB & 0b11110000) | 0b00000110;
    for (int i = 0; i < t; i++) {
        _delay_ms(1);
    }
}

void drive_backward_delay(int t) {
    PORTB = (PORTB & 0b11110000) | 0b00001001;
    for (int i = 0; i < t; i++) {
        _delay_ms(1);
    }
}

void turn_left_delay(int t) {
    PORTB = (PORTB & 0b11110000) | 0b00001010;
    for (int i = 0; i < t; i++) {
        _delay_ms(1);
    }
}

void turn_right_delay(int t) {
    PORTB = (PORTB & 0b11110000) | 0b00000101;
    for (int i = 0; i < t; i++) {
        _delay_ms(1);
    }
}

```